

CSC 4553

Fall 2026 - Day 9

Admin

- Don't forget to sign up for assignment 1 review
<https://calendar.app.google/71iRFEG6RNLqf9P29>

Interactive scheduling algorithms

Questions to consider

- What are the key differences between batch and interactive scheduling algorithms?
- How does Round Robin improve response time compared to SJF, and what tradeoff does it introduce?
- Why might a system benefit from using multiple scheduling queues rather than a single algorithm?

Why batch algorithms aren't enough

- Batch algorithms optimize for throughput, turnaround, and waiting time
- But what if a *user* is at the keyboard?
 - You click a button (e.g., while a long batch job is running) you wait 3 seconds for any response
 - A web server needs to respond to thousands of short requests, not just finish jobs quickly
- Metric to focus on: **response time**: time from request submission to first response
- Batch algorithms ignore this entirely

Batch vs. interactive scheduling

Batch Scheduling

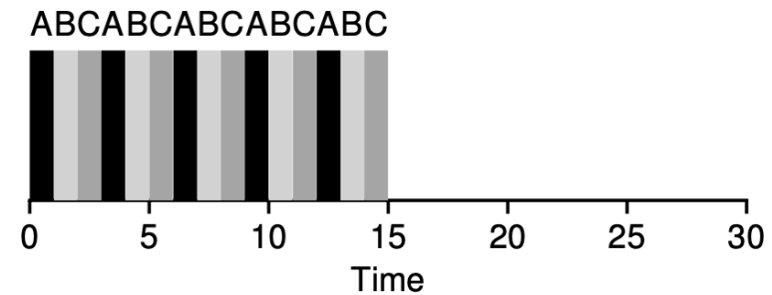
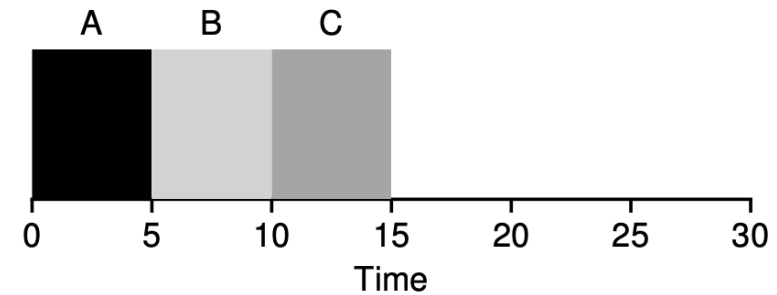
- Optimizes for: throughput, turnaround time, waiting time
- Context: data centers, HPC, background jobs
- Users don't interact with running jobs

Interactive Scheduling

- Optimizes for: response time, fairness
- Context: desktop/laptop systems, servers
- Users expect quick feedback to their actions

What about response time?

- SRTF is good if we know job lengths and we're only worried about turnaround / waiting time
- SJF: poor response
- RR: better response

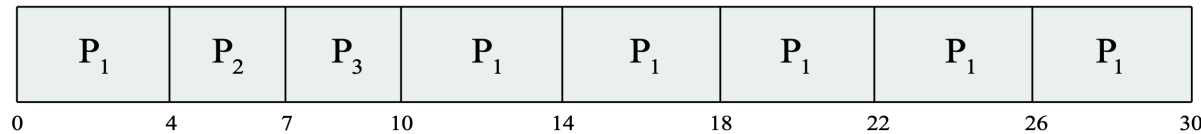


Interactive algorithm: Round Robin (RR)

- FCFS with preemption
- Based on a **time quantum** (time slice)
 - Typically 10-100 milliseconds
 - How do we pick a quantum?
 - Very short: a lot of context switching (need a large quantum, otherwise overhead is too much to be worth it)
 - Very long -> becomes FCFS: long wait times, long turnaround times
 - Circular queue
 - New processes are added to the tail
 - If a process doesn't complete during its quantum, context switch and added to the tail

RR example

- Average waiting time:



Process Burst Time

P1 24

P2 3

P3 3

Quantum = 4

$$[(10 - 4) + 4 + 7] = 17/3 = 5.66$$

- Pros:
 - RR is *fair*, but do we really want fair???
 - Typically, higher average turnaround than SJF, but better response
- Cons?
 - Long average waiting times

Gaming RR

You're working on an important project and need to maximize your CPU time. You don't care about fairness (or your colleagues). **How could a programmer "cheat" RR to get more than their fair share of the CPU?**

FCFS, SJF, SRTF, RR

- All have downsides
- Those that optimize turnaround / wait, can harm response time
- Those that optimize response time, can harm turnaround, wait
- **What should we do?**
 - Use multi-level scheduling algorithms

What isn't clear?

Comments? Thoughts?

Multi-level scheduling algorithms

Questions to consider

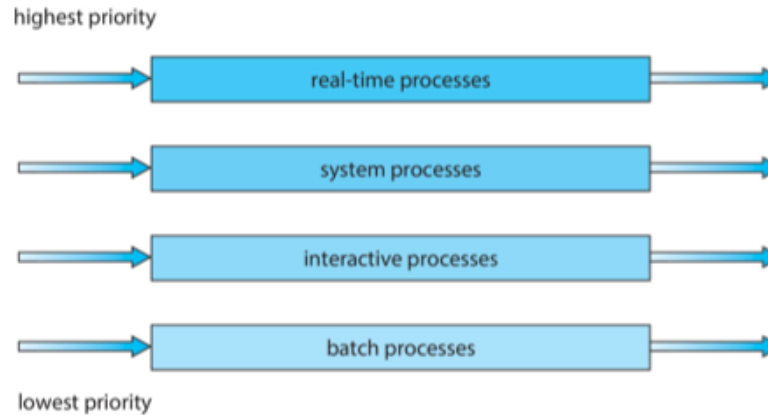
- What problem does MLFQ solve that MLQ does not, and what mechanism enables it?
- How can a process game MLFQ's original rules, and what rule change prevents it?
- How does MLFQ handle starvation of long-running processes?

Interactive algorithms: Multi-Level Queue (MLQ)

- We have classes of process, with different needs. Why not multiple schedulers satisfying those needs?
- Multilevel queue scheduler defined by the following parameters:
 - # of queues
 - Scheduling algorithms for each queue
 - Method used to determine which queue a process will enter when that process needs service
 - Scheduling among the queues

MLQ

- How do we schedule this?
 - Strict priority: low priority could starve
 - Typical: time slice among queues (e.g., real-time get 50%, system gets 30%, interactive 15%, batch 5%)
 - Each queue can implement a separate scheduling policy
- MLQ cons:
 - Starvation
 - Inflexibility



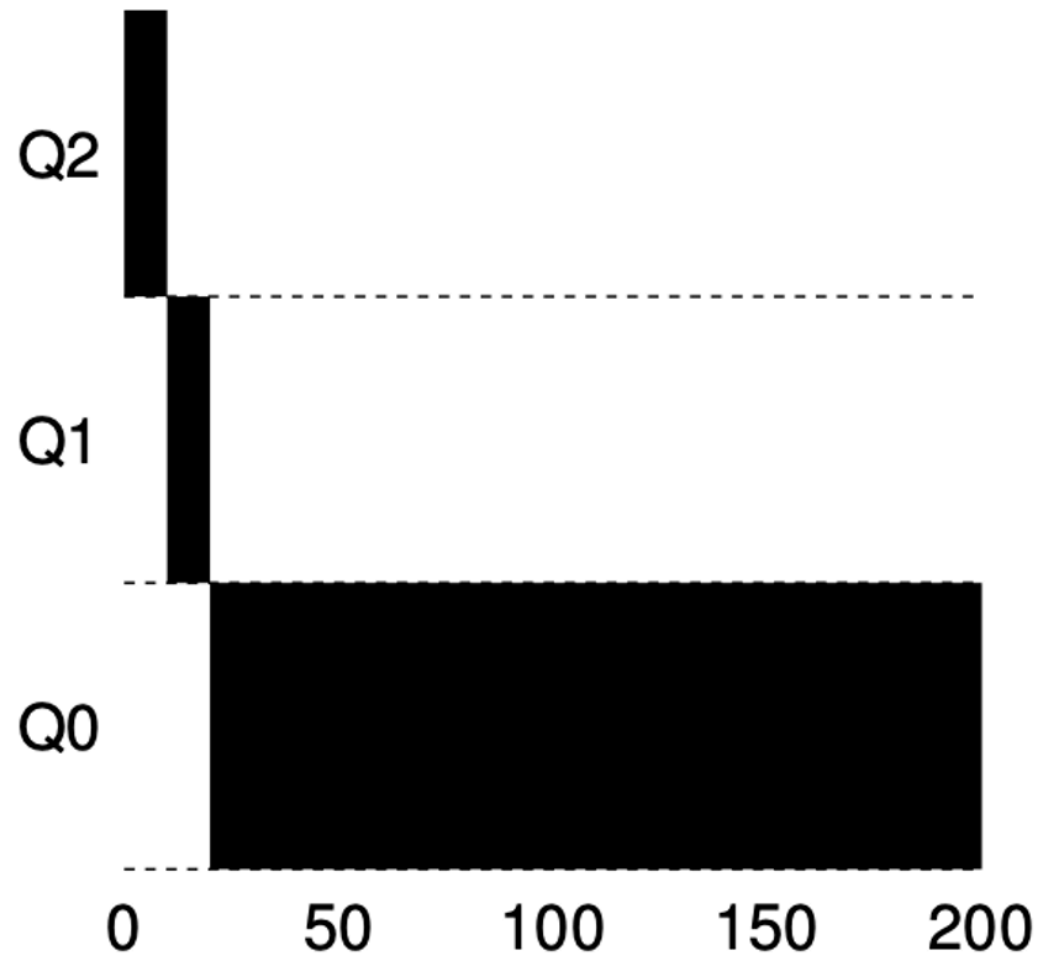
Interactive algorithms: Multi-Level Feedback Queue (MLFQ)

- A process can **move between the various queues** (goal is to avoid starvation present in MLQ)
- Metrics for movement:
 - Process requirements (we serve fast processes quickly)
 - Over consumption
 - Change in priority
 - Age

MLFQ rules

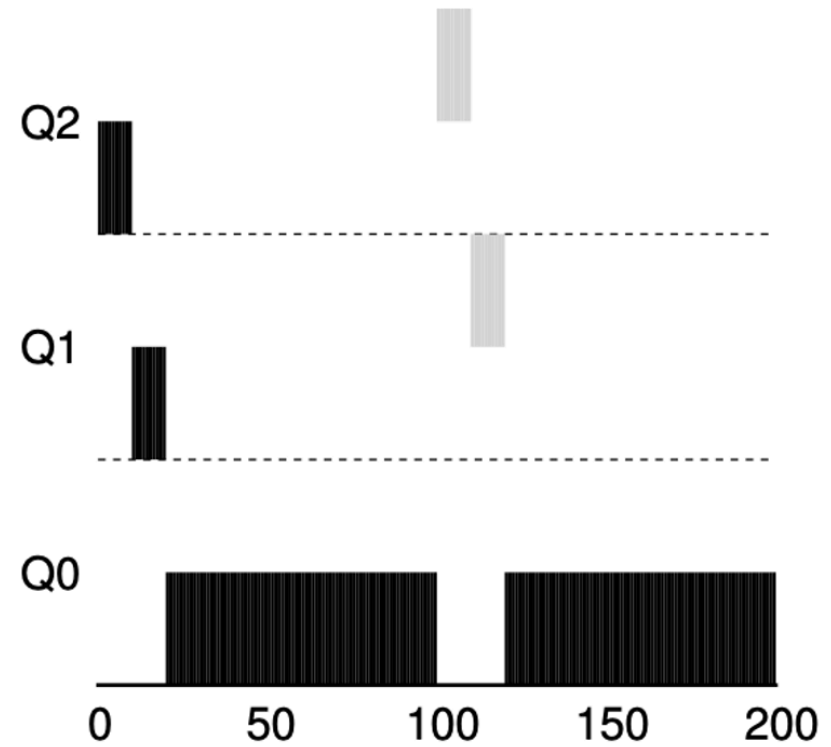
- Rule 1: If $\text{Priority}(A) > \text{Priority}(B)$, A runs (B doesn't)
- Rule 2: If $\text{Priority}(A) = \text{Priority}(B)$, A & B run in RR
- Rule 3: When a job enters the system, it is placed at the highest priority (the topmost queue)
- Rule 4a: If a job uses up its allotment while running, its priority is reduced (i.e., it moves down one queue)
- Rule 4b: If a job gives up the CPU (for example, by performing an I/O operation) before the allotment is up, it stays at the same priority level (i.e., its allotment is reset)
- Any problems with these???

MLFQ example: one process



MLFQ example: two processes

- Scenario 1: Perfectly fine



MLFQ: problem with gaming

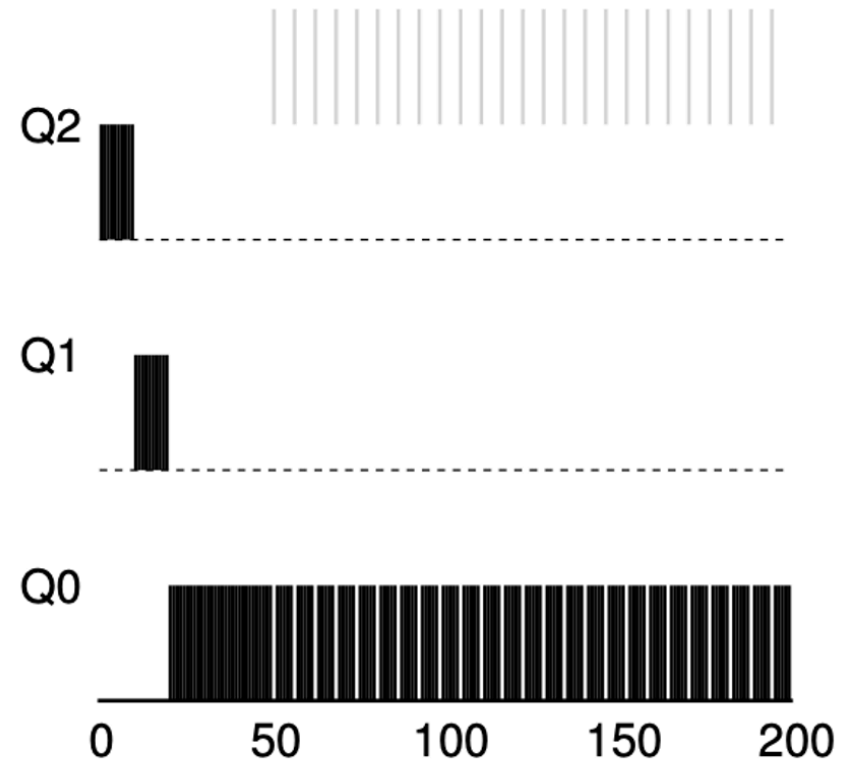
Rule 4b: If a job gives up the CPU before its allotment expires, it stays at the same priority level (allotment is reset).

Using only this rule, describe a strategy a malicious process could use to **stay in the top-priority queue indefinitely**.

Then: what rule change would stop it?

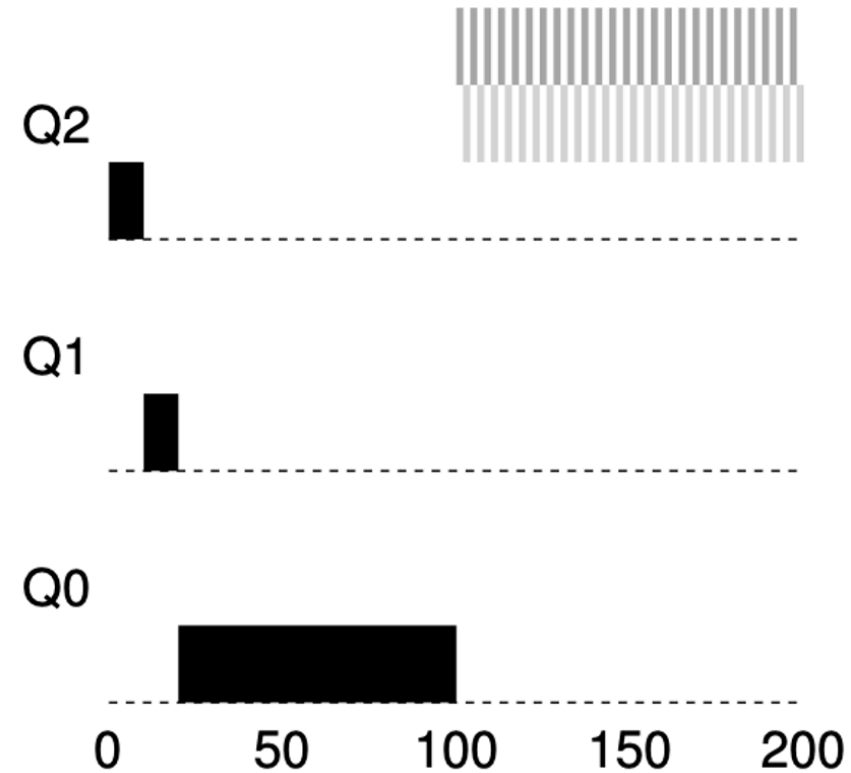
MLFQ example: two processes

- Scenario 1: Perfectly fine
- Scenario 2: programmer can game the algorithm to remain at high priority



MLFQ example: two processes

- Scenario 1: Perfectly fine
- Scenario 2: programmer can game the algorithm to remain at high priority
- Scenario 3: multiple small processes can lead to starvation



MLFQ: solving gaming

- Rule 4 (new): Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue)

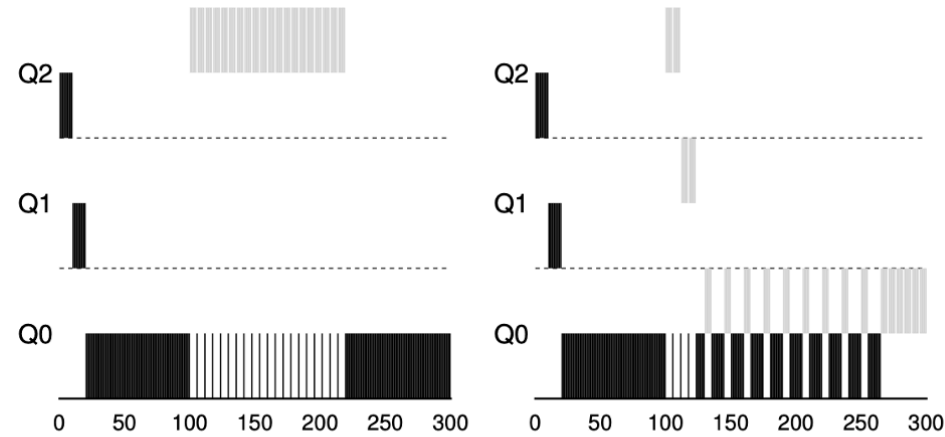
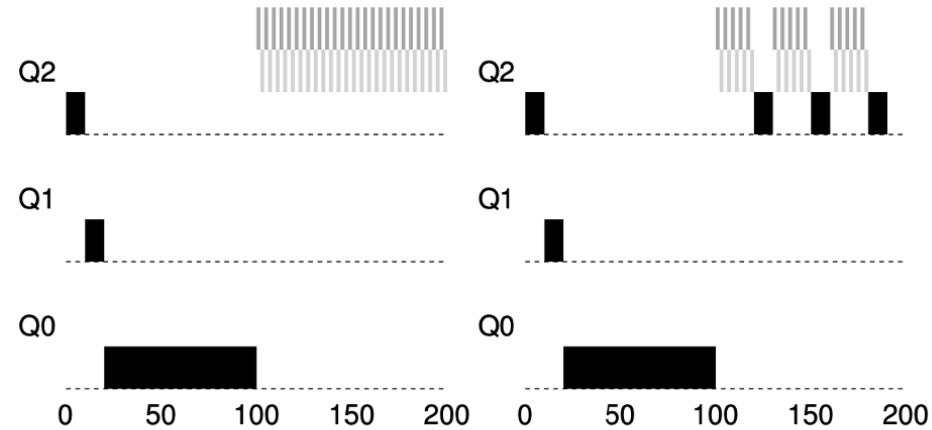


Figure 8.5: **Without (Left) and With (Right) Gaming Tolerance**

MLFQ: how can we avoid starvation?

MLFQ: solving starvation

- Rule 5: After some time period, move all the jobs in the system to the topmost queue
 - Solves starvation
 - Bonus: if a long running process evolves to be more interactive, it gets promoted



What scheduler should we use?

- Ticket booking system
 - FCFS: fairness
- Print server
 - SJF could make sense
- Web server
 - RR: fairness and short response time
- General operating systems
 - MLFQ: need to handle a mix

Linux scheduling

- Completely Fair Scheduler (CFS)
- Fair-share scheduling (see lottery and stride scheduling in the text if you want to learn more)
- Based on **vruntime** (how long any process has run): lowest **vruntime** runs next
- Divide **sched_latency** by # processes to determine a *dynamic* quantum (floor set by **min_granularity**)
 - Why do we need a floor?

Linux CFS

- Red-Black tree includes only running (or ready) processes
- Balanced such that operations are $O(\log n)$
- Very straightforward to find the next job (smallest **vruntime**) to run
- Sleeper fairness: processes waiting on I/O are given “credit” for their wait time
- **People are actively working on new features**

Controlling priority: **nice**

- Linux lets users influence CFS priority via **nice** values: **[−20,19]**
 - Lower = higher priority (more CPU weight), default = 0
- CFS scales **vruntime** accumulation by weight: high-nice processes accumulate **vruntime** faster, so they run less
- Examples:
 - **nice -n 19 ./video_encoder** — run encoder at lowest priority, keeps system responsive
 - **sudo nice -n -10 ./realtime_daemon** — boost a daemon (requires root for negative values)
- Try it: **top** shows NI column; **renice** adjusts a running process

What isn't clear?

Comments? Thoughts?